

AR Final Report: Augmented Kitchen

Marco Fasoli, 5611520 - Nicolò Calandra, 5648986

1 Important note

The link for the repository is provided in the Project Deliverables section. When opening the project, make sure to select the "Kitchen" scene from the Assets/Scenes folder and click "Play". If you want to visualize the simulated AR environment, open the "Kitchen_pesto" prefab in Assets/XR/Environments/

2 Introduction

The objective of this project is to develop an interactive, marker-based Augmented Reality (AR) application designed to assist users through a real-world process: preparing traditional Genovese pesto. We defined an interactive 10-step guided pipeline that users can follow in their physical kitchen environment. This allows the users to follow culinary steps hands-free, without relying on traditional text manuals or video screens.

3 Libraries and SDKs

The application was built using **Unity** and the libraries used include:

- **Unity ARFoundation:** chosen as the primary framework for AR development because it is built natively into the Unity ecosystem. ARFoundation allowed us to stay entirely within Unity's official development track, utilizing its native XR Plug-in Management and standard rendering pipelines. This was also the library that was presented more in detail during the lectures, so it was the most logical choice for our project.
- **ARKit XR Plugin:** since we did a live demonstration via an iPhone, we included the ARKit XR Plugin in order to build and export a demo application for the test device.

4 Design and Implementation

This application is built on Unity's AR Foundation with ARTrackedImageManager, that recognizes printed images and spawns a contextual UI card next to them. The card presents the relevant recipe step's title, description, an optional countdown timer, and an optional video demonstration. Each detected image is mapped to a discrete step in a linear pipeline, and the user progresses through the recipe by advancing that pipeline. The codebase is split across four responsibilities: AR detection and card lifecycle (PestoTutorialARManager), step/state management (PestoPipelineManager), per-card content and interaction (StepCardUI), and camera-facing orientation (BillboardUI).

4.1 The Interactive Process and Step Definition

Each step of the recipe is defined as a title, an explanatory description, an optional timer duration, and an optional video. In order to distinct each step, we assigned a unique image to every step. This keeps the recipe entirely data-driven: the steps can be edited or extended without altering program logic, which makes the tutorial easy to maintain and adapt to other recipes. Progression through the recipe is handled by a script that always knows which step is currently active. When the user signals they are ready to continue (by tapping the next button in the current card), the script advances the sequence. A card becomes visible only when both of two conditions are met: it corresponds to the step the user is currently on, and its reference image is being actively tracked.

4.2 The StepCard Prefab and Step-Dependent Layout

A single card template is reused for every step rather than designing a separate card for each one. When content is needed, one card is shown and then filled with whatever the active step requires, which keeps the design consistent and avoids duplication. The layout of the card adapts to the nature of the step's content, meaning that elements that are not used are hidden and the remaining elements are rearranged to fill the space.

4.3 Spawn Position Relative to the Camera

When a card first appears, it is attached to the reference image it belongs to. Binding the card to the image in this way lets its placement be expressed naturally as an offset from the image and means the card inherits the image's real-world position and angle automatically. The placement then incorporates the viewer's perspective: rather than leaving the card at whatever height the image happens to be mounted, its vertical position is adjusted to sit at the user's eye level. Also a small shift to the left is applied, so the card doesn't block the view of the reference image itself or the working area directly in front of it.

Note: for the iPhone demonstration, the card's position was adjusted to be slightly higher than eye level, to accommodate the fact that the device is held at a lower height than the user's head. While for the XR Simulation the card was placed as we said earlier.

4.4 Rotation Relative to the Camera (Billboarding)

The card continuously turns to face the viewer. This turning is constrained to the horizontal plane only: the card pivots left and right to follow the user but never tilts up or down. The reorientation is applied at the latest possible moment in each frame's update, after the camera's own movement has been resolved, which keeps the motion smooth and free of jitter.

4.5 Other Notable Choices

- **Reliable interactivity:** interactive controls on the card are managed so that their responses cannot accumulate duplicates each time a card is reused, ensuring a single tap always produces a single, predictable action.
- **Pausable timing:** the countdown timer can be started and paused, with its display and button label kept in sync, giving the user direct control over time-based steps such as resting or simmering.
- **Consistent physical sizing:** cards are scaled to appear at a sensible real-world size relative to the tracked image, so the interface feels physically grounded in the environment rather than arbitrarily large or small.

5 Limitations

We used a marker-based approach to ensure the system’s reliability and ease of testing, but this also means the user must have printed reference images in the real world to use the application. This could be extended for tracking kitchen tools such as salt shakers or mortar, which would make the experience more seamless and less dependent on external materials. The application is also currently designed for a specific recipe, so while the underlying system is flexible, adapting it to new recipes would require creating new reference images and step definitions.

6 Project Deliverables

6.1 Code Repository

The Unity project, containing all source code scripts, scene states, and pre-configured asset bundles, can be accessed via the following repository link:

`https://github.com/Faasol/AR-Project`

6.2 Demonstration Video

Two video demonstrations were recorded to showcase the application in action. The first video was captured inside the Unity XR Simulation and can be seen here:

`https://github.com/Faasol/AR-Project/blob/main/ARVideo.mp4`

While the second video was recorded during a live demonstration on an iPhone device, showcasing the real-world application of the AR system:

`https://github.com/Faasol/AR-Project/blob/main/ARVideoIOS.mp4`

Note that the second video just shows the core of this application, in fact video demonstrations were not included.